

Week 3 — Control Flow

Code the Dream · Python Intro

Session Overview

Explore — ~30 min

- if / elif / else
- Comparison and boolean operators
- Truthy and falsy values
- GitHub workflow for assignments

Apply — ~30 min

- Code-along: letter grades
- Code-along: age categories
- Code-along: day planner starter

Programs That Make Decisions

Every program you've written runs the same way every time.

This week that changes — you'll write programs that **respond differently** based on what the user enters or what values are in your variables.

if / else

A block runs only when its condition is `True` .

```
score = 84

if score >= 90:
    print("A")
else:
    print("Not an A")
```

elif — More Than Two Branches

Use `elif` to check multiple conditions in order.

```
if score >= 90:  
    print("A")  
elif score >= 80:  
    print("B")  
elif score >= 70:  
    print("C")  
else:  
    print("Below C")
```

Check for Understanding #1

Given `score = 75`, what does this print?

```
if score >= 90:  
    print("A")  
elif score >= 80:  
    print("B")  
elif score >= 70:  
    print("C")  
else:  
    print("F")
```

A) "A" B) "B" C) "C" D) "F"

Comparison Operators

Operator	Meaning	Example	Result
<code>==</code>	equal to	<code>5 == 5</code>	True
<code>!=</code>	not equal	<code>5 != 3</code>	True
<code><</code>	less than	<code>3 < 5</code>	True
<code>></code>	greater than	<code>5 > 3</code>	True
<code><=</code>	less or equal	<code>5 <= 5</code>	True
<code>>=</code>	greater or equal	<code>6 >= 5</code>	True

Boolean Operators

Combine conditions with `and`, `or`, `not`.

```
age = 20
has_id = True

if age >= 18 and has_id:
    print("Access granted")

if not has_id:
    print("ID required")
```


Truthy and Falsy Values

Python treats some values as `False` even without an explicit comparison.

Falsy: `0`, `""`, `[]`, `{}`, `None`

Truthy: anything else

```
name = input("Enter your name: ")

if name:
    print(f"Hello, {name}!")
else:
    print("Please enter your name.")
```

Check for Understanding #2

Which expression evaluates to `True` ?

- A) `not True and False`
- B) `0 or ""`
- C) `True and not False`
- D) `[] or {}`

Combining Conditions

Use `and` to check a range.

```
age = int(input("Enter your age: "))

if age >= 13 and age <= 17:
    print("You are a teen.")
elif age < 13:
    print("You are a child.")
else:
    print("You are an adult.")
```

GitHub Workflow — New This Week

Every assignment from here on is submitted via pull request.

Step	Command
Create branch	<code>git checkout -b assignment-3</code>
Stage changes	<code>git add .</code>
Commit	<code>git commit -m "add grade script"</code>
Push	<code>git push origin assignment-3</code>
Open PR	GitHub → Pull requests → New

Check for Understanding #3

What is the correct order of Git steps to submit an assignment?

- A) `commit` → `add` → `push` → open PR
- B) `add` → `commit` → `push` → open PR
- C) `push` → `add` → `commit` → open PR
- D) `add` → `push` → `commit` → open PR

Time to Apply

Let's put conditionals to work in the assignment.

Mentor 2 takes over

Assignment 3 Overview

Part 1 — Four warmup scripts

- `warmup1.py` — letter grades with if/elif/else
- `warmup2.py` — age categories with `input()` and `and`
- `warmup3.py` — boolean expression practice
- `warmup4.py` — sign and parity (two separate if blocks)

Part 2 — Day Planner mini-project

- Takes a day + time of day as input
- Returns a suggested activity for each combination
- Must cover at least 9 combinations + a fallback message

Code-Along 1: Letter Grades

Print a letter grade based on a hardcoded score.

```
score = 84

if score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
    print("C")
elif score >= 60:
    print("D")
else:
    print("F")
```


Code-Along 2: Age Categories

Use `input()` with `and` to check age ranges.

```
age = int(input("Enter your age: "))

if age >= 0 and age <= 12:
    print("Child")
elif age >= 13 and age <= 17:
    print("Teen")
elif age >= 18 and age <= 64:
    print("Adult")
else:
    print("Senior")
```

Code-Along 3: Day Planner Starter

Normalize input, then build a condition tree.

```
day = input("What day is it? ").lower()
time = input("What time of day? ").lower()

if day == "monday" and time == "morning":
    print("Tackle your hardest task first!")
elif day == "friday" and time == "evening":
    print("Time to unwind – great week!")
else:
    print("I don't have a suggestion for that. Try: Monday, Tuesday...")
```

Extension Challenge

Ungraded extension: Add a third input — a "mood" (e.g., "focused", "social", "tired") — and use it in your suggestions.

Your program now responds to **three variables** at once.

How does adding a third input affect the number of combinations you'd need to handle?

Wrap-Up

Today you learned:

- `if` / `elif` / `else` for branching logic
- Comparison and boolean operators
- Truthy and falsy values
- The GitHub PR submission workflow

This week: Complete warmups 1–4 and the Day Planner. Submit via pull request.

Next week: Lists, dictionaries, and loops — working with collections of data.